

Estrategia de Testes e Exigencias dos Stakeholders

Objetivo

Este documento define como os testes do MLX Pilot serao conduzidos, quais exigencias dos stakeholders precisam ser verificadas e em qual ordem novos testes devem ser criados. A estrategia prioriza feedback rapido para desenvolvimento, cobertura das areas de maior risco e um gate de release claro para evitar regressao funcional, operacional ou de seguranc

A base atual foi validada localmente em 2026-05-14 com:

- cargo test --workspace --no-fail-fast: passou com 160 testes.
- node --test providers/unified_adapter/unified_adapter.test.ts: passou com 6 testes.
- cd apps/desktop-ui && npm run test:e2e:channels-smoke: passou com 2 testes.
- cd apps/desktop-ui && npm run test:e2e:skills-smoke: passou com 1 teste.
- cd apps/desktop-ui && npm run test:e2e:agent-control-plane: passou com 1 teste.
- cd apps/desktop-ui && npm run test:e2e:agent-workspace-ui: passou com 13 testes.

Escopo do Sistema

Os testes devem cobrir os principais subsistemas do projeto:

- Backend Rust: daemon HTTP, config, catalogo, canais, agent API e runtime doctor.
- Runtime agentico: agent loop, prompt builder, sessions, memoria, audit, approval e policy.
- Ferramentas locais: read/write/edit/list/glob/grep/exec, sandbox e checkpoints.
- Skills: parser de SKILL.md, discovery, requisitos, compatibilidade e integridade.
- Providers e adapters: MLX, llama.cpp, Ollama, HTTP provider e adaptador TypeScript unificado.
- UI desktop: app Tauri com frontend estatico, Agent Workspace, Control Plane, canais, skills e console.
- Release gate: smoke operacional, carga curta, migracao/rollback, crash recovery, seguranc

Ficam fora do gate rapido local os testes que dependem de hardware, ambiente externo ou empacotamento por plataforma, como MLX real em Apple Silicon, bundles Tauri por sistema operacional, assinatura de instaladores e validacoes manuais com modelos grandes.

Matriz de Exigencias

Stakeholder	Exigencia	Evidencia de aceite
Usuario final	App inicia, daemon responde, modelos aparecem e chat/agent funcionam	Smoke UI e API em /health, /models, /chat e /agent/run
Produto/UX	Fluxos principais sao claros e recuperaveis em erro	Testes de UI para provider offline, ausencia de modelos, erro de canal e secret mascarado
Seguranc	Sandbox, approvals, audit e secrets nao vazam dados sensiveis	Testes de policy, vault, audit redaction, egress, paths sensiveis e comandos perigosos
Operacoes/release	Release so sai com estabilidade minima comprovada	Gate com carga curta, crash recovery, migracao/rollback, security audit e dry-run
Engenharia	Feedback rapido, reproduzivel e com baixa flakiness	Comandos canonicos, mocks locais, temp dirs isolados e testes regressivos por bug
Integracoes	Providers e canais preservam contratos estaveis	Testes de normalizacao, fallback, erros canonicos, timeouts e formatos de payload

Camadas de Teste

1. Unitarios rapidos

Devem rodar em toda mudanca local e em PR. Cobrem regras puras e contratos pequenos:

- agent-core: agent loop, prompt budget, sessions, memoria, approvals, audit, policy e registry.
- agent-tools: sandbox, exec, file tools, glob/grep/list e checkpoints.
- agent-skills: frontmatter, requisitos, discovery, compatibilidade e limites de prompt.
- daemon: config, normalizacao de modelos, secrets vault, canais e helpers de API.
- providers Rust: serializacao, runtime overrides e regras locais de provider.

Padrao de criacao:

- Preferir temp dirs e mocks locais.
- Evitar rede real, modelos reais e estado global persistente.
- Criar teste regressivo sempre que um bug for corrigido.
- Nomear o teste pelo comportamento observado, nao pela funcao interna.

2. Contratos de API e providers

Devem validar os endpoints e adapters mais importantes sem depender de servicos reais:

- /health: retorna estado operacional e provider configurado.
- /models: lista modelos, anota compatibilidade de agent e lida com providers indisponiveis.
- /chat: roteia provider, normaliza modelo e retorna erro canonico para falhas conhecidas.
- /chat/stream: emite status, thinking/answer deltas e evento final ou erro.
- /agent/run: executa fluxo basico com provider mockado.
- /agent/config, /agent/tools, /agent/skills, /agent/audit: preservam contrato e formato de resposta.
- /agent/channels/*: login, probe, resolve, send, logout, ambiguidade e erro canonico.
- /catalog/*: busca, detalhes e downloads com mocks de Hugging Face.

Para facilitar esses testes sem alterar APIs publicas, pode ser extraido um helper interno que monta o Router do daemon com AppState mockavel. A extracao deve manter `mlx_ollama_daemon::run()` como entrada publica principal.

3. UI desktop smoke

Devem continuar usando `node --test` e `jsdom`, mantendo o custo baixo:

- Agent Workspace carrega resumo, alterna abas e executa atalhos.
- Chat renderiza thinking, markdown e resposta final.
- Provider selector agrupa local/cloud corretamente.
- Secrets salvos aparecem mascarados e so sao revelados por acao explicita.
- Control Plane salva configuracoes e renderiza runtime health.
- Canais permitem adicionar, autenticar, resolver alvo, enviar, deslogar e exibir logs.
- Estados de erro exibem mensagem acionavel quando daemon/provider/canal falha.

4. Release gate completo

O gate completo deve ser usado antes de release e pode ser mais lento que o gate rapido. Ele deve cobrir:

- Validacao real de canais bridge/webhook com servidores mockados.
- Carga curta e concorrencia em endpoints criticos.
- Migracao, upgrade e rollback sem perda de estado.

- Auditoria pratica de seguranca.
- Recuperacao de falha ou restart.
- Dry-run de release e checklist de artefatos.

Antes de tornar node scripts/release-gate/run-all.mjs obrigatorio em CI, corrigir dois pontos:

- Trocar o repoRoot fixo de macOS em scripts/release-gate/_lib.mjs por uma resolucao dinamica baseada no diretorio do script ou em variavel de ambiente.
- Evitar escrita em arquivos versionados por padrao. O comando deve escrever em um diretorio temporario ou aceitar RELEASE_GATE_REPORT_DIR, e somente atualizar docs/release-gate-report.* quando solicitado.

Comandos Canonicos

Gate rapido local

Rodar antes de abrir PR ou ao concluir uma alteracao relevante:

```
cargo test --workspace --no-fail-fast
node --test providers/unified_adapter/unified_adapter.test.ts

cd apps/desktop-ui
npm run test:e2e:channels-smoke
npm run test:e2e:skills-smoke
npm run test:e2e:agent-control-plane
npm run test:e2e:agent-workspace-ui
```

Gate focado por area

Usar durante desenvolvimento para reduzir tempo de feedback:

```
cargo test -p mlx-agent-core
cargo test -p mlx-agent-tools
cargo test -p mlx-agent-skills
cargo test -p mlx-ollama-daemon
node --test providers/unified_adapter/unified_adapter.test.ts
```

Gate de release

Usar somente depois dos ajustes de portabilidade e saida de relatorio:

```
node scripts/release-gate/run-all.mjs
```

Enquanto esses ajustes nao forem feitos, o release gate pode ser executado manualmente, mas seu resultado nao deve ser usado como unico criterio automatizado em Windows ou CI.

Ambientes

Desenvolvimento local

- Sistema principal esperado: Windows com PowerShell.
- Requisitos: Rust estavel, Node 22 ou superior para testes TypeScript com node --test, dependencias npm do desktop instaladas.
- Estado de teste deve usar temp dirs, portas dinamicas e mocks.

CI de PR

- Deve rodar o gate rapido local.

- Não deve exigir modelos reais, credenciais, MLX, Ollama ou llama.cpp instalados.
- Deve falhar em qualquer teste unitário, contrato ou smoke de UI quebrado.

Release

- Deve rodar o gate rápido e o gate completo.
- Pode incluir runners específicos por plataforma para Tauri build.
- Validações de hardware e provedores reais devem ser registradas como checklist com ambiente, modelo, comando, resultado e responsável.

Critérios GO/NO-GO

Uma mudança comum de PR e GO quando:

- O gate rápido passa.
- Não há regressão conhecida sem teste regressivo planejado.
- Não há alteração de contrato sem documentação e teste correspondente.
- Falhas externas estão isoladas por mock ou marcadas explicitamente como fora do gate rápido.

Uma release e GO quando:

- Gate rápido e gate completo passam.
- Não há finding de segurança alto ou crítico aberto.
- Migração, restart e rollback não perdem estado.
- Não há crash sob carga curta nos endpoints críticos.
- Canais pendentes possuem resultado explícito.
- Dry-run de release foi concluído ou checklist foi validado.

Uma release e NO-GO quando qualquer uma destas condições ocorrer:

- Teste obrigatório falha.
- Security audit encontra risco alto ou crítico sem mitigação.
- Segredo aparece em arquivo, log, audit ou payload público.
- Migração, restart ou rollback causa perda de estado.
- Endpoint crítico trava, entra em timeout sistêmico ou apresenta erro alto sob carga curta.
- Gate completo não gera resultado reproduzível.

Backlog de Criação dos Testes

Prioridade 1: providers e adapters

- Expandir providers/unified_adapter/unified_adapter.test.ts para cobrir erros de payload, argumentos inválidos de tool call, respostas vazias e provider hint desconhecido.
- Adicionar testes Rust para timeout e erro canônico nos providers MLX, llama.cpp, Ollama e HTTP provider usando comandos/servidores mockados.
- Cobrir fallback entre provider primário e secundário quando o modelo não existe ou o provider está indisponível.
- Validar normalização de modelos decorados, aliases locais e famílias chat-only/tool-ready.

Prioridade 2: daemon/API

- Criar testes de contrato para /health, /models, /chat e /chat/stream com estado mockado.
- Cobrir /agent/run com provider OpenAI-compatible mockado e sem rede externa.
- Cobrir /agent/config para persistência, migração de settings legadas e preservação de defaults seguros.
- Cobrir /agent/audit e /agent/audit/export garantindo mascaramento de secrets e formato estável.

- Cobrir canais com bridge/webhook mockado para sucesso, auth error, provider error, invalid target, rate limit e ambiguidade.
- Cobrir catalogo com servidor HTTP fake para busca, detalhes, download criado, download inexistente e erro upstream.

Prioridade 3: UI desktop

- Adicionar smoke para daemon indisponivel com mensagem de erro clara.
- Adicionar caso de provider sem modelos instalados mantendo UI estavel.
- Adicionar caso de erro em /agent/run preservando input do usuario e exibindo falha.
- Adicionar caso de channel send com erro canonico e log visivel.
- Adicionar caso de secret salvo, mascarado, revelado e nao reenviado como vazio.
- Adicionar caso de streaming interrompido ou evento de erro.

Prioridade 4: release gate e operacao

- Tornar repoRoot dinamico em scripts/release-gate/_lib.mjs.
- Permitir diretorio de relatorio configuravel, sem escrever em docs/ por padrao.
- Adicionar modo --no-write-docs ou variavel equivalente para CI.
- Registrar comandos de reproducao gerados conforme plataforma atual.
- Separar resultados de smoke rapido, carga, seguranca, migracao e dry-run para diagnostico mais direto.

Prioridade 5: validacoes manuais e hardware

- Definir checklist para MLX em Apple Silicon com modelo pequeno e modelo grande.
- Definir checklist para Ollama local com modelo instalado e provider offline.
- Definir checklist para llama.cpp gerenciado pelo daemon.
- Definir checklist de Tauri build por Windows, macOS e Linux.
- Registrar hardware, sistema, versao do modelo, tempo de resposta e falhas observadas.

Politica de Manutencao

- Todo bug corrigido em API, provider, seguranca, canal, skill, tool ou UI deve receber teste regressivo no mesmo PR.
- Toda nova rota deve ter pelo menos um teste de contrato de sucesso e um de erro.
- Toda nova permissao ou capability deve ter teste de allow e deny.
- Toda integracao externa deve ter mock local para o gate rapido.
- Snapshots e relatorios versionados so devem mudar quando a mudanca for intencional e revisada.
- Teste flaky deve ser corrigido ou isolado; nao deve ser simplesmente ignorado sem issue ou plano de remediacao.